RES POLYTECHNIC, BANGALORE-91

Department of Computer Science and Engineering

MODEL ANSWER BOOKLET

Data Structure with Python

CIE Theory Test I - Complete Solutions

Program	Computer Science and Engineering
Semester	III
Course Code	25CS33I
Duration	90 minutes
Maximum Marks	50 (answer one full question from each section)
Course Coordinator	Vinay Kumar

Contents

Q. No.	Topic	Marks
1 (a)	List vs Tuple; scenario for tuple	5
1 (b)	Cube evens, add 5 to odds	10
1 (c)	Most expensive item from list of tuples	10
2 (a)	Set operations; union, intersection, difference	5
2 (b)	Students scoring above class average	10
2 (c)	Set operations function returning a tuple	10
3	Code analysis - write the output (nested stack)	25
4 (a)	Fruit collection program	Q4 total 25
4 (b)	Text analysis of Dr. Kalam paragraph	(split not given)

Note: All programs in this booklet were executed and the outputs shown are the actual outputs. A set prints its elements in no fixed order, so in the exam you may write set elements in any order.

SECTION I

Q.1 (Total 25 Marks)

(a) Differentiate between a list and a tuple [5 Marks]

Differentiate between a list and a tuple in Python. Give a real-life scenario where you would prefer a tuple over a list.

Point	List	Tuple
Definition	Ordered collection of items that can be changed.	Ordered collection of items that cannot be changed.
Mutability	Mutable - items can be added, removed or modified.	Immutable - items cannot be modified after creation.
Syntax	Square brackets: `[10, 20, 30]`	Round brackets: `(10, 20, 30)`
Methods	Many: append(), insert(), remove(), pop(), sort(), reverse()	Only two: count() and index()
Speed / Memory	Slower, uses more memory.	Faster, uses less memory.
Use as dictionary key	Not allowed (not hashable).	Allowed (hashable, if items are hashable).
Best used for	Data that changes (shopping list, marks list).	Fixed data (coordinates, date of birth).

Real-life scenario where a tuple is preferred

Storing the latitude and longitude of the college (or the date of birth of a person).

- The location of the college is a fixed value; it should not change accidentally in the program.
- A tuple is immutable, so its values are protected from modification: `college = (12.9716, 77.5946)`.
- Tuples are faster and use less memory than lists.
- A tuple can be used as a dictionary key, e.g. to store the name of a place for each (latitude, longitude).

(b) Cube of even, add 5 to odd [10 Marks]

Write a Python function that takes a list of integers and returns a new list where every even number is replaced by its cube and every odd number is increased by 5.
Example: Input: [2, 3, 4, 7, 8, 9] Output: [8, 8, 64, 12, 512, 14]

Logic (Algorithm)

1. Define a function `transform_list(numbers)` and create an empty list `result`.
2. Take each number n from the list using a **for** loop.
3. If `n % 2 == 0` (even), append `n ** 3` to result.
4. Otherwise (odd), append `n + 5` to result.
5. After the loop ends, return the new list `result`.

Program

```
def transform_list(numbers):
    result = []
    for n in numbers:
        if n % 2 == 0:
            result.append(n ** 3)     # even -> cube
        else:
            result.append(n + 5)      # odd  -> add 5
    return result


data = [2, 3, 4, 7, 8, 9]
print("Input :", data)
print("Output:", transform_list(data))
```

Output

```
Input : [2, 3, 4, 7, 8, 9]
Output: [8, 8, 64, 12, 512, 14]
```

Dry run

Element	Even / Odd	Rule applied	Result
2	Even	2 ** 3	8
3	Odd	3 + 5	8
4	Even	4 ** 3	64
7	Odd	7 + 5	12
8	Even	8 ** 3	512
9	Odd	9 + 5	14

Alternative (one line): `[n**3 if n % 2 == 0 else n + 5 for n in numbers]` *(list comprehension).*

(c) Most expensive item [10 Marks]

Write a Python function that takes a list of tuples, each containing the name of an item and its price, and returns the name of the most expensive item.
Example: Input: [('Pen', 20), ('Book', 150), ('Bag', 600), ('Ruler', 15)] Output: 'Bag'

Logic (Algorithm)

1. Define the function `most_expensive(items)`.
2. Take the first tuple as the costliest so far: store its name and price in `name, price`.
3. Visit every tuple `(n, p)` of the list.
4. If `p > price`, update `name` and `price` with n and p.
5. After the loop, return `name`.

Program

```
def most_expensive(items):
    name, price = items[0]          # assume first item is the costliest
    for n, p in items:
        if p > price:               # a costlier item is found
            name, price = n, p
    return name


items = [('Pen', 20), ('Book', 150), ('Bag', 600), ('Ruler', 15)]
print("Input :", items)
print("Output:", most_expensive(items))
```

Output

```
Input : [('Pen', 20), ('Book', 150), ('Bag', 600), ('Ruler', 15)]
Output: Bag
```

Dry run

Tuple visited	Price > current max?	name	price
('Pen', 20)	start value	Pen	20
('Book', 150)	150 > 20: Yes	Book	150
('Bag', 600)	600 > 150: Yes	Bag	600
('Ruler', 15)	15 > 600: No	Bag	600

Alternative: `max(items, key=lambda t: t[1])[0]`. *The function assumes the list is not empty.*

OR

Q.2 (Total 25 Marks)

(a) Set operations; union, intersection and difference [5 Marks]

Explain the common operations that can be performed on a set in Python. Show with examples how to find the union, intersection and difference of two sets.

Definition

A **set** is an **unordered** collection of **unique** (no duplicate) elements written inside curly brackets, e.g. `s = {1, 2, 3}`. Sets are mutable, do not support indexing, and an empty set is created with `set()` (not `{}`).

Common operations on a set

Operation	Method / Operator	Description
Add one element	`s.add(x)`	Adds x to the set
Add many elements	`s.update(iterable)`	Adds all elements of the iterable
Remove element	`s.remove(x)`	Removes x; error if x is not present
Remove safely	`s.discard(x)`	Removes x; no error if x is not present
Remove arbitrary	`s.pop()`	Removes and returns an arbitrary element
Remove all	`s.clear()`	Makes the set empty
Length / membership	`len(s)`, `x in s`	Number of elements; checks presence
Copy	`s.copy()`	Returns a copy of the set
Subset / superset / disjoint	`issubset()`, `issuperset()`, `isdisjoint()`	Relationship tests between two sets
Union, intersection, difference, symmetric difference	`| & - ^` or methods	Mathematical set operations (shown below)

Example of basic operations

```
s = {1, 2, 3}
s.add(4);       print("After add(4)      :", s)
s.remove(2);    print("After remove(2)   :", s)
s.discard(10);  print("After discard(10) :", s)   # no error
s.update({5, 6}); print("After update      :", s)
print("Length :", len(s))
print("3 in s :", 3 in s)
s.clear();      print("After clear()     :", s)
```

Output

```
After add(4)      : {1, 2, 3, 4}
After remove(2)   : {1, 3, 4}
After discard(10) : {1, 3, 4}
After update      : {1, 3, 4, 5, 6}
Length : 5
3 in s : True
After clear()     : set()
```

Union, intersection and difference

- **Union** (`A | B` or `A.union(B)`): all elements present in A or B, without duplicates.
- **Intersection** (`A & B` or `A.intersection(B)`): elements common to both A and B.

- **Difference** (`A - B` or `A.difference(B)`): elements present in A but not in B.
- **Symmetric difference** (`A ^ B`): elements present in either set but not in both.

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

print("Union A | B           :", A | B)    # A.union(B)
print("Intersection A & B    :", A & B)    # A.intersection(B)
print("Difference A - B      :", A - B)    # A.difference(B)
print("Difference B - A      :", B - A)
print("Symmetric difference  :", A ^ B)    # A.symmetric_difference(B)
```

Output

```
Union A | B           : {1, 2, 3, 4, 5, 6}
Intersection A & B    : {3, 4}
Difference A - B      : {1, 2}
Difference B - A      : {5, 6}
Symmetric difference  : {1, 2, 5, 6}
```

Venn diagrams of the set operations

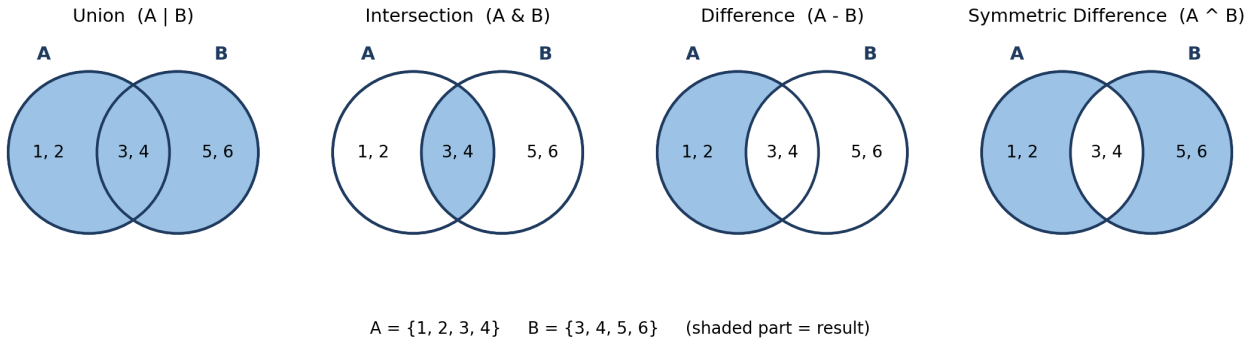


Fig. 1: Set operations on A = {1, 2, 3, 4} and B = {3, 4, 5, 6}

(b) Students above class average [10 Marks]

Write a Python function that takes a list of tuples (student name, marks) and returns a list of names of the students who scored above the class average.
Example: Input: [('Ravi', 72), ('Sita', 85), ('Arun', 60), ('Meena', 91)] Output: ['Sita', 'Meena']

Logic (Algorithm)

1. Define `above_average(students)`; set `total = 0`.
2. Add the marks of every student to `total` using a for loop.
3. Calculate `average = total / len(students)`.
4. Create an empty list `result`; loop again over all tuples.
5. If `marks > average`, append the student's name to `result`.
6. Return `result`.

Program

```
def above_average(students):
    total = 0
    for name, marks in students:
        total = total + marks
    average = total / len(students)         # class average

    result = []
    for name, marks in students:
        if marks > average:
            result.append(name)
    return result


data = [('Ravi', 72), ('Sita', 85), ('Arun', 60), ('Meena', 91)]
print("Output:", above_average(data))
```

Output

```
Output: ['Sita', 'Meena']
```

Dry run

Total = 72 + 85 + 60 + 91 = 308; Average = 308 / 4 = **77**

Student	Marks	Marks > 77 ?	Added to result
Ravi	72	No	-
Sita	85	Yes	Sita
Arun	60	No	-
Meena	91	Yes	Meena

(c) Common, symmetric difference and second-minus-first [10 Marks]

Write a Python function that takes two sets and returns a tuple containing: (i) elements common to both sets, (ii) elements present in either set but not in both, (iii) elements present in the second set but not in the first set. Example: set1 = {10, 20, 30, 40}, set2 = {30, 40, 50, 60, 70} Output: ({30, 40}, {10, 20, 50, 60, 70}, {50, 60, 70})

Logic

1. Common elements: **intersection** - `set1 & set2`.
2. Present in either but not both: **symmetric difference** - `set1 ^ set2`.
3. Present in second but not first: **difference** - `set2 - set1`.
4. Return the three results together as a tuple.

Program

```
def set_operations(set1, set2):
    common = set1 & set2           # intersection
    not_common = set1 ^ set2       # symmetric difference
    only_second = set2 - set1      # difference (set2 - set1)
    return (common, not_common, only_second)


set1 = {10, 20, 30, 40}
set2 = {30, 40, 50, 60, 70}
print("Output:", set_operations(set1, set2))
```

Output

```
Output: ({30, 40}, {10, 20, 50, 60, 70}, {50, 60, 70})
```

Python may display the elements inside each set in a different order, because sets are unordered; the elements are the same.

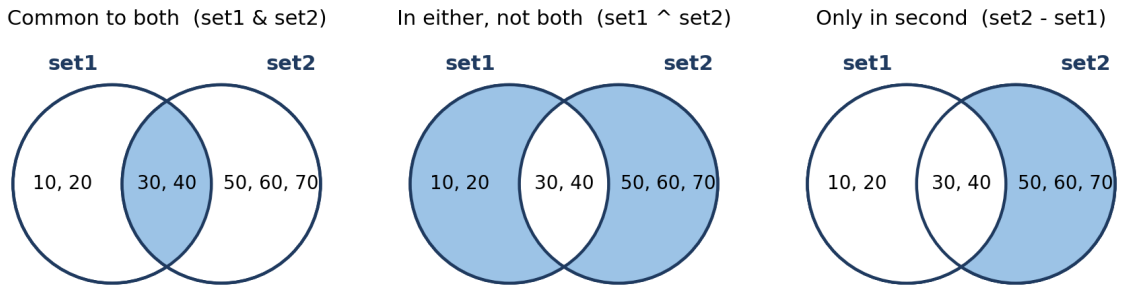


Fig. 2: The three results for set1 = {10, 20, 30, 40} and set2 = {30, 40, 50, 60, 70}

SECTION II

Q.3 (25 Marks)

Analyze the code (function sum_nested_elements with a stack and print statements) for nested_list = [2, (4, 6), [1, [3, 5]], 7] and write the output.

Code

```
def sum_nested_elements(nested_structure):
    total_sum = 0
    stack = [nested_structure]
    print(stack)
    while stack:
        current = stack.pop()
        print(current)
        for item in current:
            if isinstance(item, (list, tuple)):
                print(item)
                stack.append(item)
                print(stack)
            elif isinstance(item, (int, float)):
                total_sum += item
    return total_sum
nested_list = [2, (4, 6), [1, [3, 5]], 7]
total = sum_nested_elements(nested_list)
print(f"Total sum of elements: {total}")
```

Output

```
[[2, (4, 6), [1, [3, 5]], 7]]
[2, (4, 6), [1, [3, 5]], 7]
(4, 6)
[(4, 6)]
[1, [3, 5]]
[(4, 6), [1, [3, 5]]]
[1, [3, 5]]
[3, 5]
[(4, 6), [3, 5]]
[3, 5]
(4, 6)
Total sum of elements: 28
```

Check: 2 + 4 + 6 + 1 + 3 + 5 + 7 = 28

OR

Q.4 (Total 25 Marks)

(a) Fruit collection program

Create a collection named fruits: Apple, Banana, Mango, Orange, Grapes, Banana, Apple, Guava, Mango, Papaya, Apple, Orange, Banana, Mango, Pineapple, Cherry, Kiwi. (1) Ask the user for a fruit name and count how many times it appears. (2) Ask for a fruit name and find its position. (3) Get the first 5 fruit names. (4) Find the unique fruit names. Suggest the suitable Python concept and justify.

Suitable Python concept

List for the collection (with the list methods `count()`, `index()` and slicing), and **set** to find the unique names.

Justification

- The collection contains **duplicate** names (Apple, Banana, Mango) and the order of fruits matters - a list is **ordered** and allows duplicates; a set or dictionary cannot store duplicates in this way.
- `fruits.count(name)` directly gives the number of occurrences.
- A list is **indexed**, so `fruits.index(name)` gives the position (index starts from 0).
- **Slicing** `fruits[:5]` gives the first five items.
- A **set** keeps only unique elements, so `set(fruits)` removes duplicates in one step.

Program

```
fruits = ["Apple", "Banana", "Mango", "Orange", "Grapes", "Banana", "Apple",
          "Guava", "Mango", "Papaya", "Apple", "Orange", "Banana", "Mango",
          "Pineapple", "Cherry", "Kiwi"]

# 1. Count occurrences of a fruit
name = input("Enter a fruit name to count: ").strip().title()
print(name, "appears", fruits.count(name), "time(s) in the collection.")

# 2. Position of a fruit
name = input("Enter a fruit name to find its position: ").strip().title()
if name in fruits:
    i = fruits.index(name)                  # first occurrence
    print(name, "is at index", i, "-> position", i + 1)
else:
    print(name, "is not present in the collection.")

# 3. First 5 fruit names
print("First 5 fruits:", fruits[:5])

# 4. Unique fruit names
unique_fruits = set(fruits)
print("Unique fruits  :", sorted(unique_fruits))
print("Total unique   :", len(unique_fruits))
```

Sample output (inputs: Mango, Guava)

```
Enter a fruit name to count: Mango
Mango appears 3 time(s) in the collection.
Enter a fruit name to find its position: Guava
Guava is at index 7 -> position 8
First 5 fruits: ['Apple', 'Banana', 'Mango', 'Orange', 'Grapes']
Unique fruits  : ['Apple', 'Banana', 'Cherry', 'Grapes', 'Guava', 'Kiwi', 'Mango',
                  'Orange', 'Papaya', 'Pineapple']
Total unique   : 10
```

If a fruit is not present, `count()` *gives 0, and the* ***if name in fruits*** *check prints a 'not present' message instead of raising an error.*

(b) Text analysis

"Dr. A. P. J. Abdul Kalam was born on 15 October 1931 in Rameswaram. He served as the 11th President of India from 2002 to 2007 and worked at ISRO and DRDO." Create a Python program to: (1) measure the length of the text; (2) count the number of words; (3) count the occurrence of the stop words of, the, in, to, and; (4) convert Kalam and Rameswaram to uppercase and display the text; (5) find numeric and alphanumeric words. Suggest the suitable Python concept and justify.

Suitable Python concept

String and its built-in methods, together with a **list** (from split()) and a **dictionary** (to store stop-word counts).

Justification

- The data is text, so string methods are the natural tool.
- `len(text)` gives the number of characters (length).
- `text.split()` breaks the text into a list of words; `len(words)` gives the word count.
- `lower()` and `strip()` make the comparison with stop words case- and punctuation-independent; a dictionary stores one counter for each stop word.
- `replace()` converts the required words to uppercase.
- `isdigit()`, `isalpha()` and `isalnum()` identify numeric and alphanumeric words.

Program

```
text = ("Dr. A. P. J. Abdul Kalam was born on 15 October 1931 in Rameswaram. "
        "He served as the 11th President of India from 2002 to 2007 and "
        "worked at ISRO and DRDO.")

# 1. Length of the text (characters including spaces)
print("Length of text :", len(text))

# 2. Number of words
words = text.split()
print("Number of words:", len(words))

# 3. Stop-word count
counts = {"of": 0, "the": 0, "in": 0, "to": 0, "and": 0}
for w in words:
    w = w.strip(".,").lower()
    if w in counts:
        counts[w] += 1
for stop, c in counts.items():
    print("Count of '" + stop + "' :", c)

# 4. Convert Kalam and Rameswaram to uppercase
new_text = text.replace("Kalam", "KALAM").replace("Rameswaram", "RAMESWARAM")
print(new_text)

# 5. Numeric and alphanumeric words
numeric, alphanumeric = [], []
for w in words:
    w = w.strip(".,")
    if w.isdigit():
        numeric.append(w)
    elif w.isalnum() and not w.isalpha():
        alphanumeric.append(w)
print("Numeric words      :", numeric)
print("Alphanumeric words :", alphanumeric)
```

Output

```
Length of text : 155
Number of words: 32
Count of 'of' : 1
Count of 'the' : 1
Count of 'in' : 1
Count of 'to' : 1
Count of 'and' : 2
Dr. A. P. J. Abdul KALAM was born on 15 October 1931 in RAMESWARAM. He served as
the 11th President of India from 2002 to 2007 and worked at ISRO and DRDO.
Numeric words      : ['15', '1931', '2002', '2007']
Alphanumeric words : ['11th']
```

Notes: (i) The length 155 counts all characters including spaces and punctuation. (ii) Initials such as 'Dr.', 'A.', 'P.', 'J.' are counted as separate words by split(). (iii) A word is treated as alphanumeric when it contains both letters and digits ('11th'). (iv) Long output lines are wrapped here only to fit the page.